SI100B Fall 2026 Recitation 6 (A)

Lec11 · Python 对象模型、继承与进阶 OOP

SI100B 2026 Staff

嵌套列表的反转结果

输入：`[1, [2, 3], []]`

操作	结果
`L[::-1]`	`[[], [2, 3], 1]`
`deep_rev(L)`	`[[], [3, 2], 1]`

外层反转只重排当前元素，逐层反转还递归处理内部列表。

deep_rev：逐层构造结果

```
def deep_rev(L):
    result = []
    for e in L[::-1]:
        if type(e) == list:
            result.append(deep_rev(e))
        else:
            result.append(e)
    return result
```

每层建立新列表，递归结果作为一个元素追加，保留层级。

空列表与结果返回

```
L = [1, [2, 3], []]
print(deep_rev(L))  # [[], [3, 2], 1]
print(deep_rev([])) # []
print(L)            # [1, [2, 3], []]
```

空列表使循环执行 0 次，随后返回新建的 `[]`。

递归进入有限结构中的子列表；子列表的长度不一定小于父列表。

count_elem：统计所有整数

```
def count_elem(L):
    total = 0
    for e in L:
        if type(e) == list:
            total += count_elem(e)
        else:
            total += 1
    return total
```

`[1, [], [2, [3, 4]]]` 得 `4`；`[[], [[]]]` 得 `0`。

OOP 核心概念汇总

概念	含义	坐标示例
类 class / 类型 type	定义一类实例的行为	`Coordinate`
对象 object	具有身份、类型和值的实体	`p` 指向的坐标对象
实例 instance	相对于某个类而言的具体对象	`p = Coordinate(3, 4)`
属性 attribute	通过点号访问的命名成员	`p.x`、`p.move`
实例属性	由具体实例保存的数据	`p.x`、`p.y`
类属性	由类保存、可供实例查找的成员	`PointRecord.kind`
方法 method	通过对象或类调用的行为	`p.move(1, -2)`
self	普通实例方法接收的当前实例	调用中传入的 `p`
初始化方法	设置新实例的初始状态	`__init__`

OOP 关系与进阶概念汇总

概念	含义	本讲对应写法
继承 inheritance	子类复用、扩展基类行为	`class Child(Base)`
重写 overriding	子类重新定义继承的方法	子类定义 `describe`
多态 polymorphism	同一调用可得到不同类型的实现	`item.describe()`
组合 composition	一个对象持有其他对象	`group.points`
property	以属性读写形式执行方法逻辑	`@property`
类方法 / 静态方法	分别绑定类 / 不自动绑定参数	`@classmethod` / `@staticmethod`
特殊方法	接入运算符和内置操作	`__add__`、`__len__`
抽象基类 / 数据类	规定接口 / 自动生成常用方法	`ABC` / `@dataclass`

这些扩展按需要采用，普通类无需全部实现。

对象把状态与行为组织在一起

对象（object）具有身份、类型和当前状态。

类（class）定义一种对象的行为，实例保存具体状态。

对坐标对象而言，`x`、`y` 是数据属性，`move` 是操作它们的方法。

Python 的列表、字符串和整数也是对象，自定义类延续同一种对象模型。

贯穿示例：Coordinate

```
class Coordinate:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def move(self, dx, dy):
        self.x += dx
        self.y += dy

    def as_tuple(self):
        return self.x, self.y

    def describe(self):
        return f"({self.x}, {self.y})"
```

实例化、属性与方法调用

```
p = Coordinate(3, 4)
origin = Coordinate(0, 0)
print(p.x, p.y)            # 3 4
print(p.as_tuple())        # (3, 4)
print(origin.describe())   # (0, 0)
```

调用类创建实例；数据属性直接读取，方法通过括号调用。

`p` 与 `origin` 是两个实例，分别保存坐标状态。

self 与实例方法调用

```
p = Coordinate(3, 4)
p.move(1, -2)
print(p.as_tuple())  # (4, 2)
Coordinate.move(p, -1, 2)
print(p.as_tuple())  # (3, 4)
```

`p.move(1, -2)` 对应 `Coordinate.move(p, 1, -2)`。

Python 自动把 `p` 放到第一个参数位置，方法体中的 `self` 就接收这个实例。

绑定方法保存实例与函数

```
p = Coordinate(3, 4)
bound = p.move  # 取得方法，尚未调用
print(bound.__self__ is p)  # True
print(bound.__func__ is Coordinate.move)  # True
bound(1, -2)
print(p.as_tuple())  # (4, 2)
```

`bound` 是变量名；`__self__` 是方法对象的属性，保存已绑定的实例。

调用 `bound(1, -2)` 时，Python 将它保存的 `p` 作为 `self` 传入。

更新状态与返回结果

```
p = Coordinate(3, 4)
result = p.move(1, -2)
print(result)         # None
print(p.as_tuple())   # (4, 2)
print(p.describe())   # (4, 2)
```

`move` 改变实例，没有显式返回值；`as_tuple` 和 `describe` 返回计算结果。

实例方法也遵循普通函数的返回规则。

初始化负责建立有效状态

执行 `Coordinate(3, 4)` 时，新实例传给 `__init__` 的 `self`。

内容	职责
`__init__`	为新实例设置初始状态，正常返回 `None`
`self.x = x`	将本次参数保存为实例属性
普通方法	读取或更新已有实例

类可以不自行定义 `__init__`，但初始化参数必须符合实际采用的方法约定。

实例属性与类属性

```
class PointRecord:
    kind = "point"

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

`x`、`y` 保存在每个实例中；`kind` 保存在类中。

实例可以访问从类中找到的普通属性。

普通属性的查找与遮蔽

```
a = PointRecord(1, 2)
b = PointRecord(3, 4)
print(a.kind, b.kind)  # point point
a.kind = "special"
print(a.kind, b.kind)  # special point
print(PointRecord.kind)  # point
print(vars(a))  # {'x': 1, 'y': 2, 'kind': 'special'}
```

本例先查实例自身，再查类；`a.kind` 的赋值在实例中建立同名属性。

类属性中的列表由多个实例共享

```
class SharedGroup:
    points = []  # 执行类定义时创建一个列表

first = SharedGroup()
second = SharedGroup()
first.points.append(Coordinate(1, 2))
print(len(second.points))  # 1
print(first.points is SharedGroup.points)  # True
print(second.points is SharedGroup.points)  # True
```

两个实例都从类中找到同一个列表，`append` 修改这个共享对象。

属性赋值与对象修改的区别

操作	实际改变的内容	其他实例的结果
`a.kind = "special"`	在 `a` 中建立实例属性	`b.kind` 仍为 `"point"`
`first.points.append(p)`	修改类属性指向的共享列表	`second` 也看到列表变化
`first.points = []`	在 `first` 中绑定一个新列表	`second` 仍访问原列表

属性赋值改变实例中保存的引用；`append` 改变引用指向的列表内容。

字符串不能原地修改；`a.kind = "special"` 也不会改写原来的 `"point"`。

修改共享对象与重新绑定属性

```
first.points = []  # 在 first 中保存一个新列表
first.points.append(Coordinate(5, 6))
print(len(first.points), len(second.points))  # 1 1
print(first.points is SharedGroup.points)  # False
print(second.points is SharedGroup.points)  # True
print(first.points[0].as_tuple())   # (5, 6)
print(second.points[0].as_tuple())  # (1, 2)
```

`first.points = []` 建立实例属性，遮蔽从类中查找的同名属性。

`second` 继续访问原共享列表；`first` 此后访问自己的列表。

每个实例建立自己的容器

```
class PointGroup:
    def __init__(self):
        self.points = []

    def add(self, point):
        self.points.append(point)

first = PointGroup()
second = PointGroup()
first.add(Coordinate(1, 2))
print(len(first.points), len(second.points))  # 1 0
```

每组需要独立坐标列表时，在 `__init__` 内为每个实例新建列表。

内置类型与自定义类型

```
values = [7, 2.5, "hi", [1, 2], {"x": 1}]
print([type(v).__name__ for v in values])
# ['int', 'float', 'str', 'list', 'dict']
p = Coordinate(3, 4)
print(type(p) is Coordinate)  # True
```

`type(obj)` 返回对象的类型；自定义类同样可以成为对象的类型。

动态类型与名字绑定

```
value = 3
print(type(value).__name__)  # int
value = "three"
print(type(value).__name__)  # str
```

赋值让名字引用另一个对象；这里没有把原来的整数对象变成字符串。

动态类型表示运行时根据对象处理操作，名字不固定绑定某一种类型。

类本身也是对象

```
p = Coordinate(3, 4)
print(type(p) is Coordinate)  # True
print(type(Coordinate) is type)  # True
print(isinstance(Coordinate, type))  # True
print(type(int) is type)  # True
```

实例的类型是它所属的类；本例中，类对象的类型是 `type`。

`type` 是构造普通类对象的内置元类。

object 与 type 的不同角色

```
print(Coordinate.__bases__ == (object,))  # True
print(issubclass(Coordinate, object))     # True
print(type(object) is type)
 # True
print(type(type) is type)
   # True
print(issubclass(type, object))           # True
```

`object` 是类继承体系的根基类，`type` 是内置的类对象类型。

实例关系与继承关系是两种不同关系。

`class Coordinate(object):` 显式指定基类；本例省略基类时默认继承 `object`。

内置类型的继承关系

类型	本例中的直接基类
`bool`	`int`
`int`、`float`	`object`
`str`、`list`、`dict`	`object`

```
print(issubclass(bool, int))   # True
print(issubclass(float, int))  # False
print(True + 2)
  # 3
```

数值可以一起运算，不表示它们必然互为父类和子类。

精确类型与继承关系判断

```
print(type(True) is bool)       # True
print(type(True) is int)        # False
print(isinstance(True, int))    # True
print(isinstance(3, object))    # True
print(issubclass(bool, object)) # True
```

写法	判断内容
`type(x) is T`	精确类型是否为 `T`
`isinstance(x, T)`	实例是否属于 `T` 或其子类
`issubclass(S, T)`	类 `S` 是否为 `T` 的子类

isinstance 接纳列表子类

```
class PointList(list):
    pass

data = PointList([1, 2])
print(type(data) is list)       # False
print(isinstance(data, list))   # True
print(issubclass(PointList, list))  # True
```

允许子类型时使用 `isinstance`；精确限定类型时使用 `type(...) is ...`。

若只接受普通整数，应明确排除 `bool`，或精确检查 `int`。

类型注解与运行时检查

```
def twice(value: int) -> int:
    return value * 2

print(twice(3))     # 6
print(twice("hi"))  # hihi
```

`value: int` 声明参数预期为整数；`-> int` 声明返回值预期为整数。

Python 默认不强制检查这些注解，也不自动转换类型。字符串支持乘整数，所以第二次调用返回 `"hihi"`。

静态检查分析源代码中的类型约定

将上一页代码保存为 `examples/type_check_demo.py`，使用 mypy 检查：

```
python -m mypy examples/type_check_demo.py
```

对 `twice("hi")` 的诊断内容为：

```
Argument 1 to "twice" has incompatible type "str";
expected "int"  [arg-type]
```

mypy 分析代码而不执行函数。它报告类型不匹配，但不会让 Python 自动抛出异常。

显式检查在调用时拒绝不合法的值

```
def twice_checked(value: int) -> int:
    if not isinstance(value, int):
        raise TypeError("value must be an int")
    return value * 2

print(twice_checked(3))  # 6
try:
    twice_checked("hi")
except TypeError as error:
    print(type(error).__name__, str(error))
# TypeError value must be an int
```

`if` 检查实际参数，`raise` 抛出异常；注解本身不执行这两步。

静态检查与显式运行时检查的对照

比较内容	静态检查	显式运行时检查
执行时机	主动运行工具或编辑器检查	调用执行到判断语句时
执行主体	mypy、Pyright 等工具	Python 执行函数中的代码
判断依据	类型注解与代码推断	实际值与写出的验证条件
检查写法	`value: int` 等注解	`isinstance`、`if`、`raise`
不符合时	报告类型诊断	按代码抛出异常或处理输入
对原例的结果	标记 `twice("hi")` 不匹配	没有显式判断时仍得 `hihi`

静态检查帮助提前发现问题；显式检查在实际调用中落实输入约束。

继承规则汇总

方面	规则与边界
继承方向	子类可使用基类成员，基类不自动获得子类新增成员
方法继承	子类未重写时，沿 MRO 查找已有实现
方法重写	子类提供同名实现，基类的原实现仍然存在
初始化	未重写时可继承；重写后按需显式调用基类初始化
实例状态	属性在具体实例上建立，继承不复制另一实例的数据
类属性	子类可沿继承链查找，同名赋值可形成遮蔽
类型关系	子类实例属于基类，基类实例不自动属于子类

下文使用同一个 Coordinate 定义，逐项展示这些规则。

继承复用已有类型的行为

```
class PlainCoordinate(Coordinate):
    pass

plain = PlainCoordinate(3, 4)
plain.move(1, -2)
print(plain.as_tuple())  # (4, 2)
print(isinstance(plain, Coordinate))  # True
```

子类没有重写初始化和方法时，沿继承链使用已有实现。

子类扩展状态并重写行为

```
class NamedCoordinate(Coordinate):
    def __init__(self, x, y, name="P"):
        super().__init__(x, y)
        self.name = name

    def describe(self):
        return f"{self.name}: {super().describe()}"
```

子类新增 `name`，复用坐标初始化，并提供自己的 `describe`。

继承的方法与子类方法协同工作

```
named = NamedCoordinate(3, 4, "A")
print(named.describe())  # A: (3, 4)
named.move(1, -2)
print(named.describe())  # A: (4, 2)
print(named.as_tuple())  # (4, 2)
```

`move` 和 `as_tuple` 来自基类，`describe` 使用子类版本。

它们接收同一个实例，读取或修改同一组坐标属性。

各类方法与属性的继承行为

成员种类	子类使用时的行为	对应示例
实例方法	绑定实际子类实例作为 `self`	`move`、`as_tuple`
初始化方法	可继承；重写后不自动追加基类初始化	`__init__`
类方法	经子类访问时绑定子类作为 `cls`	`from_text`
静态方法	沿继承链取得函数，不自动绑定首参	`is_pair`
property	继承 getter、setter 等属性访问逻辑	`radius`、`x`
特殊方法	继承相应的运算或内置操作协议	`__add__`、`__len__`
抽象方法	未完成的抽象要求继续保留，完成后可实例化	抽象接口 `move`

子类可以重写这些成员；后续小节给出类方法、property 和协议的完整代码。

super 复用同一个实例上的基类实现

```
named = NamedCoordinate(3, 4, "A")
print(vars(named))  # {'x': 3, 'y': 4, 'name': 'A'}
print(type(named) is NamedCoordinate)  # True
print(super(NamedCoordinate, named).describe())
# (3, 4)
```

`super()` 返回查找代理，不会额外创建一个父类实例。

在当前单继承例子中，它找到 `Coordinate` 的相应方法。

子类中的方法调用写法

方法体中的写法	查找与调用的含义
`self.move(1, -2)`	从实际对象查找方法，可使用子类重写版本
`super().describe()`	从当前类之后沿实际 MRO 继续查找
`Coordinate.describe(self)`	直接选用指定基类的实现，实例仍是 `self`
`super().__init__(x, y)`	在当前实例上执行查找到的初始化方法

在单继承例子中，`super()` 通常找到直接基类的方法。

多继承时，显式指定基类可能跳过其他协作层；后文用 MRO 展示差异。

重写初始化时显式完成初始化链

```
class IncompleteCoordinate(Coordinate):
    def __init__(self, name):
        self.name = name

incomplete = IncompleteCoordinate("A")
try:
    incomplete.as_tuple()
except AttributeError:
    print("x and y are not initialized")
```

子类定义自己的 `__init__` 后，基类初始化不会自动再执行一次。

方法重写与动态分派

```
def describe_all(points):
    return [point.describe() for point in points]

items = [Coordinate(1, 2), NamedCoordinate(3, 4, "B")]
print(describe_all(items))
# ['(1, 2)', 'B: (3, 4)']
```

相同的调用表达式 `point.describe()`，根据实际对象找到相应实现。

方法选择取决于对象的实际类型，而不是变量名 `point`。

子类型保持调用约定

方法	基类与子类共同遵守的约定
`move(dx, dy)`	接收偏移量并更新坐标
`as_tuple()`	返回包含两个坐标值的元组
`describe()`	返回描述当前对象的字符串

重写可以扩展行为，但调用方仍应能按原接口使用对象。

`NamedCoordinate.describe()` 保留返回类型，并增加名称信息。

父子类成员与类型关系

```
parent_point = Coordinate(1, 2)
child_point = NamedCoordinate(3, 4, "A")
print(child_point.as_tuple())  # (3, 4)，继承的方法
print(child_point.name)        # A，子类新增属性
print(hasattr(parent_point, "name"))  # False
print(isinstance(child_point, Coordinate))  # True
print(isinstance(parent_point, NamedCoordinate))  # False
print(issubclass(NamedCoordinate, Coordinate))    # True
print(issubclass(Coordinate, NamedCoordinate))    # False
```

子类实例具有基类行为；单独创建的基类实例不自动获得子类的 `name`。

鸭子类型依赖对象提供的行为

```
class ForeignCoordinate:
    def describe(self):
        return "external coordinate"

print(describe_all([ForeignCoordinate()]))
# ['external coordinate']
print(isinstance(ForeignCoordinate(), Coordinate))
# False
```

`describe_all` 只依赖 `describe()`，不强制要求某个共同父类。

组合把对象保存在另一个对象中

```
group = PointGroup()
group.add(Coordinate(1, 2))
group.add(NamedCoordinate(3, 4, "A"))
print([p.describe() for p in group.points])
# ['(1, 2)', 'A: (3, 4)']
group.points[0].move(1, 0)
print(group.points[0].as_tuple())  # (2, 2)
print(isinstance(group, Coordinate))  # False
```

`PointGroup` 包含坐标对象；`NamedCoordinate` 则是一种坐标对象。

继承结构与方法查找

结构	类之间的关系	方法查找特点
单继承	一个直接基类	沿继承链寻找实现
多层继承	子类的基类还有基类	可使用祖先类的成员
多重继承	一个类有多个直接基类	同名方法由实际 MRO 决定
菱形继承	多个分支共享一个祖先类	C3 让共同祖先在 MRO 中只出现一次

MRO 是 Python 为类确定的查找次序，可通过 `类.__mro__` 查看。

基类书写顺序参与决定 MRO，不能把所有继承图简单当成深度优先搜索。

单继承的 MRO

```
print([cls.__name__ for cls in NamedCoordinate.__mro__])
# ['NamedCoordinate', 'Coordinate', 'object']
```

MRO（Method Resolution Order）给出类及其基类的查找次序。

对本例的普通方法，先找到子类实现，再沿 MRO 继续查找。

`super()` 从当前类之后的位置继续。

协作式多重继承的基类

```
class TraceRoot:
    def steps(self):
        return ["root"]

class LeftTrace(TraceRoot):
    def steps(self):
        return ["left"] + super().steps()

class RightTrace(TraceRoot):
    def steps(self):
        return ["right"] + super().steps()
```

菱形继承的查找次序

```
class CombinedTrace(LeftTrace, RightTrace):
    pass

combined = CombinedTrace()
print([cls.__name__ for cls in CombinedTrace.__mro__])
# ['CombinedTrace', 'LeftTrace', 'RightTrace',
#  'TraceRoot', 'object']
print(combined.steps())  # ['left', 'right', 'root']
```

Python 用 C3 线性化确定 MRO，此例共同基类只出现一次。

super 可以继续到兄弟类

```
print(super(LeftTrace, combined).steps())
# ['right', 'root']
print(TraceRoot.steps(combined))
# ['root']
```

在 `CombinedTrace` 的 MRO 中，`LeftTrace` 之后是 `RightTrace`。

因此 `super()` 不是固定指向源代码中写出的直接父类。

协作式方法调用的约定

组成	本例中的约定
方法签名	各层都接收 `self`，无额外参数
继续调用	每个协作层调用一次 `super().steps()`
结束位置	`TraceRoot.steps` 返回最终列表
搜索顺序	由实际对象类型的 MRO 决定

复杂多继承需要各参与类共同遵守调用约定。

property 提供计算型属性

```
class RadialCoordinate(Coordinate):
    @property
    def radius(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5

radial = RadialCoordinate(3, 4)
print(radial.radius)  # 5.0
radial.move(-3, 0)
print(radial.radius)  # 4.0
```

`radial.radius` 会自动执行上面的 `radius(self)`，计算并返回当前半径。

getter 指读取属性时执行的函数；这里使用 `radial.radius`，不加 `()`。

setter 在赋值时维护约束

```
class NonnegativeXCoordinate(Coordinate):
    @property
    def x(self):
        return self._x

    @x.setter
    def x(self, value):
        if value < 0:
            raise ValueError("x must be nonnegative")
        self._x = value
```

继承来的初始化对 `self.x` 赋值时，也会执行这个 setter。

公开属性 x 与内部存储 _x

使用方式	实际执行的操作
`obj.x`	getter 返回 `self._x` 中保存的值
`obj.x = 5`	setter 验证数值，再执行 `self._x = 5`
`obj._x`	直接读取普通实例属性中的值
`obj._x = -1`	直接改变存储，绕过 `x` 的 setter 检查

`_x` 是普通属性，单下划线表示内部使用约定。`x` 与 `_x` 的关联由 getter、setter 中的代码建立。

setter 内若写 `self.x = value`，会反复触发自身，最终产生 `RecursionError`。

属性赋值的完整执行结果

```
checked = NonnegativeXCoordinate(3, 4)
checked.x = 5
print(checked.x)  # 5
try:
    checked.x = -1
except ValueError:
    print("negative x rejected")
print(checked.x)  # 5
print(vars(checked))  # {'_x': 5, 'y': 4}
```

失败的赋值未更新 `_x`，实例仍保留合法的 x 值。

普通属性与 property 的查找差异

```
checked.__dict__["x"] = -100
print(checked.x)  # 5
try:
    radial.radius = 9
except AttributeError:
    print("radius is read-only")
```

在默认实例查找中，找到的 `property` 数据描述符优先于同名实例字典项。

没有 setter 的 property 通过公开接口只读，内部状态仍可由其他方法改变。

类方法与静态方法的定义

```
class ParsedCoordinate(Coordinate):
    @classmethod
    def from_text(cls, text):
        x, y = text.split(",")
        return cls(float(x), float(y))

    @staticmethod
    def is_pair(value):
        return isinstance(value, (tuple, list)) and len(value) == 2
```

`classmethod` 接收类，`staticmethod` 不自动接收实例或类。

cls 保留实际调用的子类

```
class DerivedParsedCoordinate(ParsedCoordinate):
    pass

parsed = DerivedParsedCoordinate.from_text("3,4")
print(type(parsed).__name__)  # DerivedParsedCoordinate
print(parsed.as_tuple())      # (3.0, 4.0)
print(ParsedCoordinate.is_pair((3, 4)))  # True
print(parsed.is_pair([1]))
 # False
```

`from_text` 中的 `cls` 是实际调用的类，`cls(...)` 创建该类型的实例。

三种方法的绑定内容

方法种类	自动接收的首参	本讲示例
普通实例方法	实例 `self`	`p.move(1, 2)`
类方法	实际类 `cls`	`ParsedCoordinate.from_text("3,4")`
静态方法	无	`ParsedCoordinate.is_pair((3, 4))`

实例方法处理对象状态，类方法可以构造对象，静态方法组织相关辅助计算。

这三种方法的区别在于调用时自动绑定的内容。

特殊方法接入 Python 的操作协议

常用操作	类型可实现的特殊方法
`str(p)`、`print(p)`	`__str__`
`repr(p)`、容器中的表示	`__repr__`
`a + b`	`__add__`，必要时考虑反向方法
`a == b`	`__eq__`
`len(path)`、迭代、索引	`__len__`、`__iter__`、`__getitem__`

这些方法让自定义对象与内置操作配合使用。

自定义 str 与 repr

```
class PrintableCoordinate(Coordinate):
    def __str__(self):
        return f"({self.x}, {self.y})"

    def __repr__(self):
        return f"PrintableCoordinate({self.x!r}, {self.y!r})"
```

`__str__` 面向可读展示，`__repr__` 面向明确的对象表示。

两者都必须返回字符串。

打印对象与打印对象容器

```
display_point = PrintableCoordinate(3, 4)
print(display_point)       # (3, 4)
print(repr(display_point)) # PrintableCoordinate(3, 4)
print([display_point])     # [PrintableCoordinate(3, 4)]
```

列表展示内部元素时使用元素的 `repr` 表示。

未实现 `__str__` 时，默认展示可以回退到 `__repr__`。

运算符重载：向量相加

```
class Vector(Coordinate):
    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

v = Vector(1, 2) + Vector(3, 4)
print(v.as_tuple())  # (4, 6)
```

+ 构造并返回新向量，两个输入向量保持原值。

NotImplemented 是协议返回值

```
v = Vector(1, 2)
print(Vector.__add__(v, 3) is NotImplemented)  # True
try:
    v + 3
except TypeError:
    print("unsupported addition")
```

返回 `NotImplemented` 让 Python 尝试协议中的其他实现。

双方都不支持时，最终得到 `TypeError`。

`NotImplemented` 是特殊返回值，`NotImplementedError` 是异常。

值相等与对象身份

```
class ValueCoordinate(Coordinate):
    def __eq__(self, other):
        if type(self) is not type(other):
            return NotImplemented
        return self.as_tuple() == other.as_tuple()
```

本例只比较相同精确类型的坐标值。

`==` 使用相等协议，`is` 判断是否引用同一个对象。

相等协议与哈希约束

```
a = ValueCoordinate(1, 2)
b = ValueCoordinate(1, 2)
print(a == b, a is b)  # True False
print(ValueCoordinate.__hash__ is None)  # True
try:
    hash(a)
except TypeError:
    print("unhashable value coordinate")
```

本例定义值相等后默认不可哈希，避免把可变坐标直接当作稳定的键。

为组合容器实现内置协议

```
class Path(PointGroup):
    def __len__(self):
        return len(self.points)

    def __iter__(self):
        return iter(self.points)

    def __getitem__(self, index):
        return self.points[index]
```

`Path` 继承容器初始化与 `add`，再接入长度、迭代与索引操作。

长度、迭代、索引与真假值

```
path = Path()
print(bool(path))  # False
path.add(Coordinate(1, 2))
path.add(NamedCoordinate(3, 4, "A"))
print(len(path))  # 2
print([p.as_tuple() for p in path])  # [(1, 2), (3, 4)]
print(path[0].as_tuple())  # (1, 2)
print(bool(path))  # True
```

未定义 `__bool__` 时，真假值判断可通过 `__len__` 完成。

抽象基类显式规定行为接口

```python
from abc import ABC, abstractmethod

class Movable(ABC):
    @abstractmethod
    def move(self, dx, dy):
        raise NotImplementedError
```

继承方需要提供具体的 `move` 实现，才能完成这种抽象接口的实例化要求。

`abstractmethod` 表达接口要求；这里的异常用于尚无具体实现的方法体。

未完成抽象接口的类不可实例化

```
class IncompleteMovable(Movable):
    pass

try:
    IncompleteMovable()
except TypeError:
    print("abstract method is not implemented")
print(type(Movable).__name__)  # ABCMeta
```

类定义可以完成，但缺少抽象方法的具体实现时不能创建其实例。

通过继承满足抽象接口

```
class MovableCoordinate(Coordinate, Movable):
    pass

movable = MovableCoordinate(3, 4)
movable.move(1, -2)
print(movable.as_tuple())  # (4, 2)
print(isinstance(movable, Movable))  # True
```

MRO 中 `Coordinate.move` 已提供具体实现，因而这个子类可以实例化。

抽象接口、实现复用和实际调用在同一个类中衔接。

dataclass 自动生成常用方法

```python
from dataclasses import dataclass

@dataclass
class DataCoordinate:
    x: float
    y: float

    def move(self, dx, dy):
        self.x += dx
        self.y += dy
```

默认按字段生成初始化、对象表示和相等比较等方法，业务方法仍可自己定义。

数据类的初始化、表示与相等

```
data_point = DataCoordinate(3, 4)
print(data_point)  # DataCoordinate(x=3, y=4)
print(data_point == DataCoordinate(3, 4))  # True
data_point.move(1, -2)
print(data_point)  # DataCoordinate(x=4, y=2)
print(type(data_point.x).__name__)  # int
```

字段注解不会自动把实参转换成 `float`。

默认生成的相等比较要求双方精确类型一致，再比较字段值。

default_factory 建立独立可变字段

```
from dataclasses import field

@dataclass
class DataPath:
    points: list = field(default_factory=list)

data_first = DataPath()
data_second = DataPath()
data_first.points.append(DataCoordinate(1, 2))
print(len(data_first.points), len(data_second.points))  # 1 0
```

工厂在每次初始化需要默认值时调用，避免实例共用同一个默认列表。

frozen 数据类限制字段重新赋值

```
from dataclasses import FrozenInstanceError

@dataclass(frozen=True)
class FrozenCoordinate:
    x: float
    y: float

fixed = FrozenCoordinate(3, 4)
try:
    fixed.x = 8
except FrozenInstanceError:
    print("field assignment is blocked")
```

`frozen` 限制通常的字段赋值，不等于所有内部对象都深度不可变。

把类型、继承与协议串联使用

```
route = Path()
route.add(Coordinate(1, 2))
route.add(NamedCoordinate(3, 4, "A"))
for point in route:
    point.move(1, 0)
print([p.describe() for p in route])
# ['(2, 2)', 'A: (4, 4)']
print(isinstance(route, PointGroup))  # True
print(isinstance(route, Coordinate))  # False
```

容器通过组合保存对象，继承容器行为，通过迭代协议接入 `for`。

对象关系的四个层次

关系	本讲示例
对象属于某种类型	`type(p) is Coordinate`
类本身也是对象	`isinstance(Coordinate, type)`
子类继承已有类型	`issubclass(NamedCoordinate, Coordinate)`
对象内部包含其他对象	`route.points` 保存坐标实例

方法绑定决定操作哪个实例，MRO 决定查找哪份类实现。

特殊方法让自定义类型参与 Python 的内置操作。

详细用法与扩展顺序

1. 用初始化和实例方法组织对象状态与操作。
2. 区分类属性和实例属性，理解普通查找与属性描述符。
3. 用继承复用接口与实现，用重写提供子类行为。
4. 用 MRO 和 `super()` 理解协作式方法调用。
5. 用特殊方法接入协议，用抽象类或数据类表达额外约定。

配套 Notebook 按同样顺序提供完整代码、输出和边界验证。

参考：对象、类型与继承

- 正课：教授 `Lec11_A.pdf`，第 42-68 页。
- Python 官方类教程：属性、绑定方法、继承与重写。
- Python 数据模型：对象、类型及特殊方法。
- 内置函数文档：`type`、`isinstance`、`super`。
- Helsinki MOOC · Class hierarchies：继承教学顺序。
- Python MRO HOWTO：C3 方法查找顺序。

参考：进阶对象机制

- Descriptor Guide：属性查找、方法绑定和 property。
- abc 官方文档：抽象基类与抽象方法。
- dataclasses 官方文档：字段、生成方法、默认工厂和 frozen。
- typing 官方文档：类型注解与运行时的边界。
- Harvard CS50P · Lecture 8：完整示例、继承和特殊方法的教学参考。

Thanks for Listening